

Lab 3 - Building a Simple Processor Integration & Control

In this lab series you will be designing a simple 8-bit single-cycle processor which includes an ALU, a register file and control logic, using Verilog HDL. Follow the guidelines given here to build your processor.

The microarchitecture of a processor is designed based on an Instruction Set. Your processor should implement the instructions `add`, `sub`, `and`, `or`, `mov`, `loadi`, `j` and `beq`. All instructions are of 32-bit fixed length, and should be encoded in the format shown below.

OP-CODE (bits 31-24)	RD / IMM (bits 23-16)	RT (bits 15-8)	RS / IMM (bits 7-0)
-------------------------	--------------------------	-------------------	------------------------

- Bits (31-24) : OP-CODE field identifies the instruction's operation. This should be used by the control logic to interpret the remaining fields and derive the control signals.
- Bits (23-16) : A register (RD) to be written to in the register file, or an immediate value (jump or branch target offset).
- Bits (15-8) : A register (RT) to be read from in the register file.
- Bits (7-0) : A register (RS) to be read from in the register file, or an immediate value.

Here are some examples about the usage and descriptions of these instructions:

```
add 4 1 2 (add value in register 2 to value in register 1, and place the result in register 4)
sub 4 1 2 (subtract value in register 2 from the value in register 1, and place the result in register 4)
and 4 1 2 (perform bit-wise AND on values in registers 1 and 2, and place the result in register 4)
or 4 1 2 (perform bit-wise OR on values in registers 1 and 2, and place the result in register 4)
j 0x02 (jump 2 instructions forward from the next instruction to be executed, by manipulating the
        Program Counter. Ignore bits 15-0)
beq 0xFE 1 2 (if values in registers 1 and 2 are equal, branch 2 instructions backward by manipulating
              the Program Counter)
mov 4 1 (copy the value in register 1 to register 4. Ignore bits 15-8)
loadi 4 0xFF (load the immediate value 0xFF to register 4. Ignore bits 15-8)
```

You will be building your processor in four steps:

- In Lab 2: part 1, you will build an 8-bit ALU which implements all the functional units required to support the instructions `add`, `sub`, `and`, `or`, `mov`, and `loadi`.
- In Lab 2: part 2, you will implement a simple 8x8 register file.
- **In Lab3, you will implement the control logic and integrate all the components from parts 1 and 2 together to work as a complete processor.**
- In Lab 4, you will upgrade your processor to support `j` and `beq` instructions.

Integration & Control

Now you should compose a working CPU using your ALU and Register File, supporting the instructions `add`, `sub`, `and`, `or`, `mov`, and `loadi`. To do this, you will need to implement the control logic in a top-level module (you may call this module as *cpu*). Your CPU needs an instruction fetching mechanism and a **Program Counter** (PC) register which points to the next instruction. You may choose to have a *control_unit* module and instantiate it within your top-level *cpu* module, or you may choose to implement all control logic in your *cpu* top-level module itself.

The following module definition gives a template interface for your CPU:

```
module cpu(PC, INSTRUCTION, CLK, RESET)
```

Since we do not have an instruction memory module yet to hold instructions, you should keep the instructions as an array of hardcoded instruction words (array size = 1024 bytes, i.e. 256 instructions) in the testbench file which you use to test your *cpu*. Your *cpu*'s instruction fetching mechanism should read the hardcoded instructions asynchronously from the testbench, based on the address provided by PC.

You need combinational control logic to **decode** a fetched instruction, extract the OP-CODE, source/destination registers and immediate values. Based on the OP-CODE (bits 31-26), all control signals should be generated and sent to the Register File, ALU and other components appropriately. Bits 25-0 need to be directly sent to appropriate places where they may be used, without needing to wait.

For arithmetic instructions (`add` and `sub`), assume that the operands are **signed integers** with negative values presented in Two's Complement format. You will need to perform the Two's Complement operation on the second operand before supplying it to the ALU, in order to support both `add` and `sub` instructions using the same adder functional unit. You will need to use MUXs to achieve the desired control.

Pay careful attention to how you coordinate the timings of instruction fetching, decoding, register reading, execution and register writing. To realistically simulate the latencies of instruction fetching, decoding and execution, you should include the following artificial timing delays in your CPU:

- PC Update (write to PC register) = One time unit (#1)
- Instruction Memory Read = Two time units (#2)
- Instruction Decode (generating control signals) = One time unit (#1)
- Two's complement operation = One time unit (#1)

In order to automatically increment the PC value by 4, you will need to include a dedicated adder. We will assume that this adder has a latency of one time unit (#1), which will work in parallel to instruction memory reading.

Use GTKWave (or any other similar tool that you prefer) to help you visualize the timings.

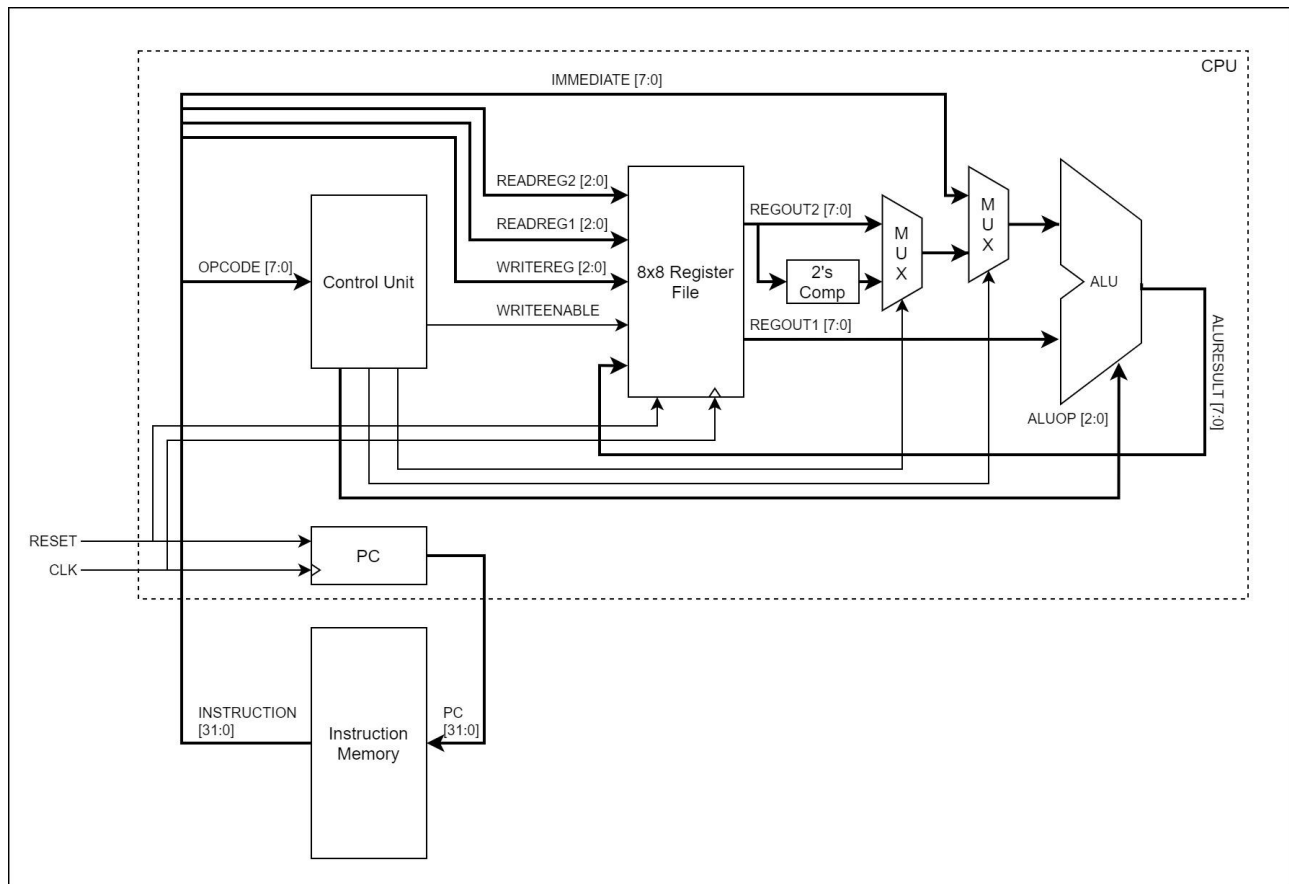


Figure 1: Overview of CPU

An overall block diagram for this simple CPU is provided in Figure 3. All components except the instruction memory should be housed within your top-level *cpu* module.

You must thoroughly test your *cpu* using several different software programs (instruction sequences). For this, you need to prepare programs as machine code and hardcode them inside your testbench file, one program at a time. Since it is easier to write textual assembly programs (rather than writing machine code), you may use the provided *CO224Assembler* tool to convert textual assembly into machine code. Note that you must add your **OP-CODE** value definitions to the *CO224Assembler.c* file before using it to generate the assembler tool. A shell script is provided to you, which can convert assembled machine code into a memory image which you can easily copy onto your testbench.

The diagram given in Figure 4 shows the worst-case timing of your single-cycle CPU for the `add`, `sub`, `and`, `or`, `mov`, and `loadi` instructions. One clock cycle spans for eight (8) time units duration, rising edge to rising edge. Every instruction should complete within one clock cycle, and any data to be written to the register file should be ready by the rising edge at the end of the clock cycle. Writing to registers and PC should be synchronized to rising edges of the clock, while reading of registers should happen asynchronously. The given timing delays should be artificially added to the corresponding operations in order to realistically simulate the latencies observable in a synthesized datapath. For the sake of simplicity, we will be assuming that our multiplexers and wires have negligible delays.

`add`:

PC Update	Instruction Memory Read		Register Read	ALU
#1	#2		#2	#2
	PC+4 Adder		Decode	
	#1		#1	
Register Write				
#1				

`sub`:

PC Update	Instruction Memory Read		Register Read	2's Comp
#1	#2		#2	#1
	PC+4 Adder		Decode	
	#1		#1	
Register Write				
#1				

`and/or/mov`:

PC Update	Instruction Memory Read		Register Read	ALU
#1	#2		#2	#1
	PC+4 Adder		Decode	
	#1		#1	
Register Write				
#1				

`loadi`:

PC Update	Instruction Memory Read		ALU	
#1	#2		#1	
	PC+4 Adder		Decode	
	#1		#1	
Register Write				
#1				

Figure 2: Timing Details for the Datapath

You must also implement the reset functionality of the CPU, so that the program can be restarted by setting the `RESET` signal high for a short period of time. If the `RESET` signal is high when writing to PC at a positive clock edge, write zero to PC instead of the next PC value in order to restart the program. So the CPU will read the instruction memory at address zero. In addition to this, your register file content should also be reset at the same time.

Note that data memory reading and writing is not implemented in Lab 3. You will be adding those functionalities in Lab 5.

1. Build the top-level *cpu* module to integrate the ALU and Register File using appropriate control logic. Include **a lot of comments**.
2. Write a testbench and thoroughly test your completed design. Hardcode your software program (instruction sequence) within the testbench file.
3. Submit a compressed file ***groupXX_lab3.zip*** containing your Verilog files with the top-level *cpu module*, *alu* and *reg_file* modules and any other Verilog files with any sub modules of your design, testbench, and screenshots of timing diagrams clearly showing the synchronized operation of the datapath and control signals for the given six instructions.

Note that any form of plagiarism will result in zero marks for the entire lab.